

Engineering Notes: LMD18200T Hardware Control

PREMIUM UNIVERSITY-LEVEL GUIDE ON H-BRIDGE PROTOTYPING, TELEMETRY, AND CIRCUIT DIAGNOSTICS

Topic Overview

This document provides a highly structured engineering analysis of hardware prototyping, embedded programming, and electrical troubleshooting using the LMD18200T H-Bridge IC paired with an Arduino Uno R4 Minima. The scope covers component-level architecture, high-speed position tracking (quadrature encoders), analog current-sensing mirrors, bootstrap capacitor dynamics, and structural safety protocols when handling mid-power DC motor loads under constrained bench environments.

Crucially, these notes incorporate specific laboratory adaptations for substitute component values (e.g., 0.1 μF bootstrap caps and a smaller 100 μF bulk capacitor) alongside advanced embedded firmware fixes designed to combat signal noise and control loop windup.

Core Concepts

1. H-Bridge Motor Regulation

An H-bridge is a switching matrix configured to reverse the polarity of voltage applied across an inductive load. The LMD18200T integrates four discrete DMOS switches configured as an H-bridge, minimizing power dissipation and enabling seamless bidirectional control of DC motors via basic digital logic inputs.

2. Bootstrap Gate Drive Mechanics

To fully activate high-side N-channel DMOS power switches, their gate voltage must exceed the rail supply voltage (V_S). The LMD18200T resolves this using an internal charge-pump combined with external bootstrap capacitors. These charge when an output is low and step up the switching voltage when the high-side switch commands power.

3. Current Mirror Telemetry

Conventional current monitoring forces power through an inline shunt resistor, generating excessive heat loss (I^2R). The LMD18200T features an integrated Current Mirror loop that outputs a tiny, proportional fraction of the total motor current:

$$\text{Sense Ratio} = 1 / 2650 \quad (377 \mu\text{A per Amp of load current})$$

This allows precise analog telemetry via standard microcontroller ADC pins without sacrificing system efficiency.

4. Contact Resistance & Joule Heating

Temporary prototyping frameworks like plastic solderless breadboards possess inherent contact resistance. When subject to high operational currents (such as startup or motor stall spikes), these high-resistance junctions produce localized heating according to Joule's Law ($P = I^2R$), creating physical hazards if left unmitigated.

Detailed Notes

1. Component Pinout & Connection Matrix

When implementing the naked 11-pin TO-220 package directly, look at the front printed label with the legs extending downward (numbered 1 to 11 left to right). The connections must be mapped explicitly as follows:

Pin	Name	Target Terminal	Functional Description
1	BOOTSTRAP 1	Capacitor 1 (Leg 1)	Gate drive voltage reservoir for Output 1.
2	OUTPUT 1	Motor Lead A / Cap 1 (Leg 2)	High-power delivery path out to the motor.
3	DIRECTION	Arduino Digital Pin 4	Polarity controller (HIGH = Forward, LOW = Reverse).
4	BRAKE	Direct Ground Rail	Must be actively pulled LOW to disengage internal brake.
5	PWM	Arduino Digital Pin 5	Input pin accepting high-frequency speed modulation signals.
6	V _S (POWER)	DC Supply Positive (+12V)	Main high-power voltage supply path for the load.
7	GND	Common Ground Hub	Common low-side return reference for logic and power.
8	CURRENT SENSE	Arduino Analog Pin A0	Outputs mirrored current across a pull-down network to GND.
9	THERMAL FLAG	Leave Disconnected	Open-collector diagnostics; drops low above 145°C.
10	OUTPUT 2	Motor Lead B / Cap 2 (Leg 2)	High-power delivery path out to the motor.
11	BOOTSTRAP 2	Capacitor 2 (Leg 1)	Gate drive voltage reservoir for Output 2.

2. Hardware Subsystems & Structural Zones

Zone 1: The High-Power Path (Solder/Crimp Mandatory)

Pins 6, 7, 2, and 10 carry the heavy motor currents.

CRITICAL SAFETY RULE: These pins must completely bypass the breadboard plastic slots. High-amperage power conduits must be joined directly using mechanical clamps, thick solid-core copper wires (≥ 22 AWG), or direct solder joints to prevent melting the breadboard tracks.

Zone 2: The Bootstrap Loops (Direct-on-Chip)

Loop A: Connected directly across Pin 1 and Pin 2.

Loop B: Connected directly across Pin 10 and Pin 11.

Note: Pins 2 and 10 share connections with the motor leads. No other external power or logic lines should splice into these loops.

Zone 3: The Common Grounding Hub

To eliminate ground loops and erratic signal references, implement a unified Common Ground Rail strategy. The following lines must meet at a single physical bus bar junction:

- 12V Power Supply Negative Terminal (-)
- LMD18200T Power Ground (Pin 7)
- Arduino Microcontroller GND Pin
- Encoder Sensor Ground Wire

Zone 4: Signal Feedback Conditioning (Pin 8 Network)

The current mirror output from Pin 8 requires a passive conditioning circuit built on the breadboard before entering Arduino Analog Pin A0:

- **Sense Resistor (R_s):** A 2.2 k Ω resistor tied from Pin 8 directly to the GND rail.
- **Low-Pass RC Filter:** A 1 k Ω series resistor flowing into Arduino Pin A0, with a 100 nF (0.1 μ F) ceramic filtering capacitor placed from Pin A0 to GND to strip away high-frequency motor switching noise.

3. Component Substitutions & The "PWM Paradox"

Bootstrap Substitution (0.1 μ F vs. 10 nF)

Standard configurations dictate 10 nF capacitors for rapid, nearly instantaneous charging cycles. When substituting 0.1 μ F (100 nF) capacitors, the volume is ten times larger.

The Paradox: Running an oversized 0.1 μF capacitor at high PWM duty cycles holds the high-side switches closed continuously. The weak internal charge pump requires significant time to fill this large capacitive well, introducing a prolonged startup delay where the motor hums silently before rotating.

Polarity Warning: If using Tantalum 0.1 μF capacitors, they are polarized. The side marked with the negative sign (-) or stripe must connect to the lower potential pin (Pin 2 for Loop A; Pin 10 for Loop B). Non-polarized ceramic discs can be oriented in either direction.

Bulk Reservoir Downsizing (100 μF Safety Bounds)

If the power rail bulk capacitor is downsized from 1000 μF to a small 100 μF capacitor, it will empty almost instantly during torque spikes. This creates massive voltage dips that drop below the brownout threshold of the Arduino, causing it to constantly reset.

The Mitigation Strategy: The system must be constrained via software, forcing an aggressive ceiling on current draw (`MAX_CURRENT_AMPS = 0.3`) and clamping the duty cycle (`MAX_PWM_LIMIT = 100`) to guarantee power rail stability.

Definitions

H-Bridge Matrix — A specialized electronic switching topology utilizing four solid-state components that coordinates voltage vector directions across an active inductive load to control rotational spin direction.

Bootstrap Gate Drive — A non-polarized or polarized capacitive pumping circuit that captures low-side energy to generate an elevated gate-drive potential higher than the main source power rail (V_S), allowing high-side N-channel MOSFETs to conduct fully.

Stiction (Static Friction) — The fundamental threshold of mechanical friction resistance that a stationary physical shaft assembly must break through before achieving kinetic rotation.

Motor Stall Current — The peak electrical current consumed when an electric motor's shaft is physically restricted from rotating while full voltage is sustained. Under this condition, the windings act purely as a low-value static resistor.

Integral Windup / Sticky Integral — A control loop phenomenon where an error cannot be cleared immediately, causing the integral term of a PI/PID controller to accumulate a massive value. On release, this excess "memory" keeps the control effort saturated, making a haptic wall feel sticky or glued.

State Resetting — A haptic control strategy where historical tracking variables (like `integralError`) are instantly wiped to zero the moment a physical boundary condition changes (e.g., exiting a virtual wall into free space), ensuring crisp transitions.

Important Formulas / Rules

1. Joule Power Dissipation Rule

$$P = I^2 \times R$$

2. Modified Current Mirror Conversion Algorithm

To extract absolute load current metrics (I_{motor}) from the 12-bit ADC stream of an Arduino Uno R4 Minima (0–4095 range over a 5.0V reference) utilizing a 2.2 k Ω (2200 Ω) pull-down resistor:

$$V_{\text{out}} = (\text{ADC_Reading} \times 5.0) / 4095.0$$

$$I_{\text{motor}} = V_{\text{out}} / (2200.0 \times 0.000377)$$

PASSIVITY GUARD RULE

In haptic boundary rendering, a virtual wall must only store or dissipate energy; it must never add energy to the system. The target current must be constrained such that the wall only pushes back against penetration:

$$\text{If } \tau_{\text{target}} > 0.0 \Rightarrow \tau_{\text{target}} = 0.0$$

Examples

1. Production-Grade Cascaded Haptic Script

This script implements a strict 1kHz execution clock running a 4x quadrature encoder state machine, a 2nd-order Butterworth velocity filter, a haptic virtual wall boundary, and an inner PI current regulator embedded with a Ghost Current Killer noise threshold and an Instant Free-Space State Reset.

```

/**
 * 1-DOF Cascaded Current-Controlled Haptic System
 * Hardware: Arduino Uno R4 Minima, LMD18200T, RS-555PH
 * Architecture: 1kHz Outer Haptic Loop -> Inner PI Current Loop
 */

// --- Pin Definitions ---
const int clkPin = 2;    // Encoder Phase A
const int dtPin = 3;     // Encoder Phase B
const int DIR_PIN = 4;   // LMD18200T Direction (Pin 3)
const int PWM_PIN = 5;   // LMD18200T PWM (Pin 5)
const int CURRENT_SENSE_PIN = A0; // LMD18200T Pin 8 via RC Filter

// --- Encoder Variables ---
volatile long encoderPos = 0;
volatile int lastEncoded = 0;

// --- Timing & Loop Control ---
unsigned long lastTimeMicros = 0;
long lastRawPosition = 0;
const unsigned long SAMPLE_INTERVAL_MICROS = 1000; // 1kHz loop clock
int telemetryCounter = 0;

// --- 2nd-Order Butterworth Filter (fc = 100Hz, fs = 1000Hz) ---
const float b0 = 0.067455;
const float b1 = 0.134911;
const float b2 = 0.067455;
const float a1 = -1.142981;
const float a2 = 0.412802;
float x_hist[2] = {0.0, 0.0};
float y_hist[2] = {0.0, 0.0};

// --- Hardware Constants ---
const float RS_OHMS = 2200.0; // 2.2k Ohm sense resistor
const float SENSE_RATIO = 0.000377; // 377 uA per Amp

// --- PI Current Controller Parameters ---
float Kp_curr = 25.0;
float Ki_curr = 500.0;
float integralError = 0.0;
const float MAX_INTEGRAL = 200.0;

// --- Haptic Physics & Safety Bounds (100uF Optimized) ---
const long WALL_POSITION_COUNTS = 400;
const float K_STIFFNESS = 0.05;
const float B_DAMPING = 0.002;
const float MAX_CURRENT_AMPS = 0.3; // Safe cap for 100uF reservoir
const int MAX_PWM_LIMIT = 100; // Protected ceiling to prevent brownouts

void setup() {
    pinMode(DIR_PIN, OUTPUT);
    pinMode(PWM_PIN, OUTPUT);
    pinMode(clkPin, INPUT_PULLUP);
    pinMode(dtPin, INPUT_PULLUP);

    analogReadResolution(12); // Initialize 12-bit mode for Uno R4

```

```

int MSB = digitalRead(clkPin);
int LSB = digitalRead(dtPin);
lastEncoded = (MSB << 1) | LSB;

attachInterrupt(digitalPinToInterrupt(clkPin), updateEncoder, CHANGE);
attachInterrupt(digitalPinToInterrupt(dtPin), updateEncoder, CHANGE);

Serial.begin(115200);
lastTimeMicros = micros();
}

void loop() {
    unsigned long currentTimeMicros = micros();

    if (currentTimeMicros - lastTimeMicros ≥ SAMPLE_INTERVAL_MICROS) {
        long currentRawPos;

        // Atomic Snapshot of Encoder Position
        noInterrupts();
        currentRawPos = encoderPos;
        interrupts();

        // 1. Velocity Computation and Butterworth Filtering
        float dt = (currentTimeMicros - lastTimeMicros) / 1000000.0;
        long deltaCounts = currentRawPos - lastRawPosition;
        float rawVelocityCounts = (float)deltaCounts / dt;

        float filteredVelocityCounts = (b0 * rawVelocityCounts) + (b1 * x_hist[0]) + (b2 *
x_hist[1])
                                - (a1 * y_hist[0]) - (a2 * y_hist[1]);

        x_hist[1] = x_hist[0];
        x_hist[0] = rawVelocityCounts;
        y_hist[1] = y_hist[0];
        y_hist[0] = filteredVelocityCounts;

        // 2. Outer Loop: Haptic Virtual Wall Boundary
        float targetCurrentAmps = 0.0;

        if (currentRawPos > WALL_POSITION_COUNTS) {
            long penetration = currentRawPos - WALL_POSITION_COUNTS;
            float springForce = K_STIFFNESS * penetration;
            float dampForce = B_DAMPING * filteredVelocityCounts;

            targetCurrentAmps = -(springForce + dampForce);

            if (targetCurrentAmps > 0.0) targetCurrentAmps = 0.0; // Passivity Guard
        } else {
            targetCurrentAmps = 0.0;
            integralError = 0.0; // Wipes memory instantly to eliminate "Sticky Wall" feel
        }

        // Safety Current Clamp
        if (targetCurrentAmps < -MAX_CURRENT_AMPS) targetCurrentAmps = -MAX_CURRENT_AMPS;
        if (targetCurrentAmps > MAX_CURRENT_AMPS) targetCurrentAmps = MAX_CURRENT_AMPS;
    }
}

```

```

// 3. Inner Loop: PI Current Regulation
float actualCurrentAmps = readMotorCurrent();
float currentError = abs(targetCurrentAmps) - actualCurrentAmps;

float pTerm = Kp_curr * currentError;

integralError += currentError * dt;
if (integralError > MAX_INTEGRAL) integralError = MAX_INTEGRAL;
if (integralError < -MAX_INTEGRAL) integralError = -MAX_INTEGRAL;
float iTerm = Ki_curr * integralError;

float controlEffort = pTerm + iTerm;
int pwmOutput = (int)controlEffort;

// 4. Safe Actuation Clamping
if (pwmOutput > MAX_PWM_LIMIT) pwmOutput = MAX_PWM_LIMIT;
if (pwmOutput < 0) pwmOutput = 0;

if (targetCurrentAmps < 0) {
    digitalWrite(DIR_PIN, LOW); // Retraction force execution
} else {
    digitalWrite(DIR_PIN, HIGH);
}
analogWrite(PWM_PIN, pwmOutput);

// 5. Downsampled Telemetry Monitor (40Hz updates)
telemetryCounter++;
if (telemetryCounter ≥ 25) {
    Serial.print("Pos:"); Serial.print(currentRawPos);
    Serial.print("    Target_A:"); Serial.print(targetCurrentAmps, 3);
    Serial.print("    Actual_A:"); Serial.print(actualCurrentAmps, 3);
    Serial.print("    PWM:"); Serial.println(pwmOutput);
    telemetryCounter = 0;
}

lastRawPosition = currentRawPos;
lastTimeMicros = currentTimeMicros;
}
}

float readMotorCurrent() {
    int rawADC = analogRead(CURRENT_SENSE_PIN);
    float voltage = (rawADC / 4095.0) * 5.0;
    float currentAmps = voltage / (RS_OHMS * SENSE_RATIO);

    // THE GHOST CURRENT KILLER: Suppresses breadboard analog floating noise up to 0.25A
    if (currentAmps < 0.25) currentAmps = 0.0;

    return currentAmps;
}

void updateEncoder() {
    int MSB = digitalRead(clkPin);
    int LSB = digitalRead(dtPin);
    int encoded = (MSB << 1) | LSB;
    int sum = (lastEncoded << 2) | encoded;

```

```

if(sum == 0b0010 || sum == 0b1011 || sum == 0b1101 || sum == 0b0100) encoderPos++;
if(sum == 0b0001 || sum == 0b0111 || sum == 0b1110 || sum == 0b1000) encoderPos--;

lastEncoded = encoded;
}

```

2. Thermal Meltdown Case Study

Consider a 12V motor stalling and drawing 1.25 Amps through a single structural breadboard contact pin that has oxidized and deformed, establishing a localized resistance of $0.5\ \Omega$:

$$P = I^2 R = (1.25)^2 \times 0.5 = 1.5625 \times 0.5 = 0.781\ \text{Watts}$$

Practical Assessment: Emitting ≈ 0.8 Watts of localized, continuous thermal energy within an unventilated, tiny plastic cavity forces immediate structural warping. This causes surrounding plastic columns to collapse, short-circuiting the adjacent power and ground rails.

Common Mistakes

- **Breadboard-Routed Power Lines:** Routing heavy motor outputs (Pins 2, 10) or 12V power supply lines through delicate breadboard slots. These tracks can warp or melt instantly at higher currents.
- **Floating Brake Configuration (Pin 4):** Leaving Pin 4 unassigned creates a floating high-impedance state. Static environmental noise will trick the chip into activating internal braking, causing unexplained stalls.
- **Integral Accumulation in Ghost Zones:** Allowing the inner loop integral accumulator to wind up while running a "Dry Run" test with the 12V supply disconnected. The loop sees zero actual current and ramps the PWM to maximum, which saturates the system.
- **Inverting Bootstrap Electrolytic Components:** Accidentally fitting polarized tantalum or electrolytic capacitors backward across Pins 1–2 or 11–10, leading to component failure when the high-side power switches cycle.

Interview / Viva Questions

Q1: Detail the physical necessity of bootstrap capacitor installations in integrated DMOS drivers like the LMD18200T.

Answer: The high-side switches in the integrated H-bridge matrix are N-channel DMOS units. To achieve full conduction, an N-channel device requires a gate voltage that is significantly higher than its source supply potential (V_S). The external bootstrap capacitor acts as an independent reservoir. It charges during low-side execution cycles and subsequently stacks its voltage on top of V_S to drive the upper gates securely into saturation.

Q2: Why is integrated current mirror sensing structurally superior to using conventional low-side inline shunt resistors?

Answer: Conventional shunt topologies place a low-value power resistor directly inline with the heavy load path, continuously wasting energy as structural heat (I^2R) and dropping the motor's terminal voltage. Current mirror frameworks reflect an identical but heavily attenuated copy ($1 / 2650$) of the active current path to a low-power signal branch. This maximizes efficiency and completely isolates delicate microcontrollers from high-power paths.

Q3: Explain how intermediate PWM speed adjustments can cause a motor assembly to stall completely even when the primary voltage supply is verified active.

Answer: Lowering the PWM duty cycle diminishes the average root-mean-square (RMS) voltage presented to the motor terminal leads. If this averaged potential translates into an electromagnetic torque vector that is smaller than the mechanical static friction parameter (stiction) of the shaft, gears, and links, the motor stalls. It consumes power and outputs stall current, but remains structurally motionless.

Q4: What is the purpose of resetting the integral error to zero the moment the encoder leaves the virtual wall?

Answer: This handles the "Sticky Integral" problem. In free space, the target current is zero. If you exit the wall fast, the integral term retains historical memory of the error accumulated inside the wall. Wiping `integralError = 0.0;` inside the else block forces the PWM to drop to zero instantly, preventing a sticky, sluggish sensation at the wall's boundary.

Quick Revision Sheet (1-Page Summary)

- **Ground Reference Unification:** High-power supply grounds and low-power microcontroller grounds must be bound together at a single common junction point to ensure a proper signal return loop.
- **Brake Override Protocol:** Pin 4 must be hardwired straight to ground to prevent random electrostatic noise from triggering automatic braking.
- **Bootstrap Selection Dynamics:** While a substituted $0.1\ \mu\text{F}$ capacitor works as a larger gate reservoir, it introduces internal charge pump lag at elevated duty cycles. Use the datasheet-optimized $10\ \text{nF}$ value to eliminate startup lag if full high-frequency scaling is required.
- **Power Separation Architecture:** Logic commands belong on breadboards; heavy power tracks belong completely outside them. Never route motor lines through delicate prototyping slots.
- **Current Conversion Formula (12-bit, $2.2\ \text{k}\Omega$):** $I = \text{AnalogVoltage} / (2200 \times 0.000377)$.
- **The Ghost Current Killer Rule:** Always set a low-level threshold filter (e.g., $< 0.25\text{A}$) in your analog current-sensing function during breadboard prototyping to prevent electrical float from confusing your PI control loop.
- **$100\mu\text{F}$ Safety Limits:** When operating with a drastically downsized power rail reservoir capacitor, always clamp the maximum output current and PWM in software to insulate your system against brownout resets.

Final Takeaways

Industrial embedded motor control requires a strict physical distinction between logic-level signal paths and high-power drive lines. Perfect code and optimal logic connections will still result in physical breakdown if high-current paths are choked by resistive prototyping hardware. Engineers must calculate thermal distributions using $P = I^2R$, manage inductive charging loops through proper bootstrap capacitor matches, and bypass fragile test tracks with heavy solid-core copper wiring to ensure safe and reliable motor operation.